Department of Computer Engineering

CENG350 Software Engineering

Software Requirements Specification for Koster Seafloor Observatory (KSO)

Group #81

By

Author Name 1

Alperen Ovak - 2580801

Author Name 2

Mithat Can Timurcan - 2581064

Sunday 4th May, 2025

Contents

List of Figures iii

List of Tables iv

1 Introduction 1

1.1 Purpose of the System 1

1.2 Scope 2

1.3 System Overview 3

1.3.1 System Perspective 3

1.3.1.1 System Interfaces 4

1.3.1.2 User Interfaces 5

1.3.1.3 Hardware Interfaces 5

1.3.1.4 Software Interfaces 6

1.3.1.5 Communication Interfaces 7

1.3.1.6 Memory Constraints 7

1.3.1.7 Operations 8

1.3.2 System Functions 9

1.3.3 Stakeholder Characteristics 10

1.3.4 Limitations 10

1.4 Definitions 12

2 References 13

3 Specific Requirements **14**

3.1 External Interfaces 15

3.2 Functions 17

3.3 Logical Database Requirements 27

3.4 System Quality Attributes 28

3.4.1 Usability Requirements 28

3.4.2 Performance Requirements 29

3.4.3 Reliability Requirements 29

3.4.4 Maintainability Requirements 30

3.4.5 Portability Requirements 30

3.5 Design Constraints 30

3.6 Supporting Information 32

4 Suggestions to Improve The Existing System **33**

4.1 System Perspective 33

4.2 External Interfaces 35

4.3 Functions 37

4.4 Logical Database Requirements 45

4.5 System Quality Attributes 46

4.5.1 Suggestions to Improve Usability Requirements 46

4.5.2 Suggestions to Improve Performance Requirements 46

4.5.3 Suggestions to Improve Reliability Requirements 47

4.5.4 Suggestions to Improve Maintainability Requirements 47

4.5.5 Suggestions to Improve Portability Requirements 47

4.6 Design Constraints 48

4.7 Supporting Information 50

List of Figures

1.1 System Context Diagram 4

3.1 External Interfaces Diagram 15
3.2 Use Case Diagram 17
3.3 Sequence Diagram for Annotate Footage 20
3.4 Activity Diagram of Fine-tuning ML Models 22
3.5 State Diagram of Running Predictions 24
3.6 Logical Database Diagram 27

4.1 Improved System Context Diagram 33
4.2 Improved External Interfaces Diagram 35
4.3 Improved UseCase Diagram 37
4.4 Activity Diagram of Detection of Endangered or Possibly Extinct Marine Species 39
4.5 Sequence Diagram of Real-Time Marine Monitoring via Livestream and Environmental Sensors 41
4.6 State Diagram of Model Processing Selection: GPU vs. TPU 43
4.7 Improved Logical Database Diagram 45

List of Tables

1 Revision History v

1.1 System Functions of Koster Seafloor Observatory (KSO) 9

3.1 Tabular Description of *Upload Underwater Footage to KSO* 18

3.2 Tabular Description of *Annotate Footage via Citizen Science* 19

3.3 Tabular Description of *Fine-tune Machine Learning Models* 21

3.4 Tabular Description of *Running Predictions on New Footage* 23

3.5 Tabular Description of *Visualize and Report Biodiversity Data* 25

3.6 Tabular Description of *API Access for External Researchers* 26

4.1 Tabular Description of *Detection of Endangered or Possibly Extinct Marine Species* 38

4.2 Tabular Description of *Real-Time Marine Monitoring via Livestream and Environmental Sensors* 40

4.3 Tabular Description of *Model Processing Selection: GPU vs. TPU* . . 42

4.4 Tabular Description of *Data Extraction for Researchers* 44

Revision History

Date	Reason For Changes	Version
04.04.2025	First Draft	1.0

Table 1: Revision History

1. Introduction

This document is the Software Requirements Specification (SRS) [1] for the open-source project Koster Seafloor Observatory (KSO). KSO is a modular, open-source software platform developed to support marine ecological research through the analysis of underwater video data. It integrates citizen science, machine learning, and high-performance computing to enable large-scale annotation, classification, and interpretation of subsea imagery. The system includes tools for data management, species identification through crowdsourced annotations, and automated biological object detection, facilitating reproducible and scalable biodiversity monitoring in marine environments.

1.1 Purpose of the System

The **Koster Seafloor Observatory (KSO)** is an **open-source** initiative designed to facilitate the efficient management, annotation, and analysis of large volumes of underwater footage for **marine ecological research**. The system integrates **citizen science and machine learning** to enhance the identification and monitoring of marine biodiversity. KSO enables researchers to collect, process, and analyze data from underwater recordings, particularly those captured in Sweden's **Kosterhavets National Park**. By leveraging **public participation** and **high-performance computing**, the system enhances the efficiency and scalability of marine biodiversity assessments.

KSO's purpose includes:

- **Data Management**: Storing and archiving subsea footage for research purposes.
- **Citizen Science Participation**: Enabling volunteers to annotate video clips to

support marine species identification.

- **Machine Learning Integration**: Using citizen-provided classifications to train AI models for automated species recognition.
- **Ecological Research Support**: Providing scientists with analytical tools to monitor marine biodiversity and assess environmental changes over time.

By **combining open-source tools, citizen science, and AI-based automation**, KSO aims to enhance the accessibility and accuracy of marine biodiversity research.

1.2 Scope

The **Koster Seafloor Observatory (KSO)** is a **modular software system** that focuses on the collection, classification, and automated analysis of underwater imagery. Its primary application is within **marine ecological research**, specifically for monitoring **cold-water coral reefs** and other benthic habitats in the **Kosterhavets National Park**.

The system is structured into three core modules:

1. **Data Management Module** – Facilitates the storage and processing of large-scale underwater video footage, ensuring **long-term archiving** and accessibility for researchers.
2. **Citizen Science Module** – Engages volunteers to classify and annotate marine species using a **web-based platform**, enhancing data collection efficiency.
3. **Machine Learning & High-Performance Computing Module** – Utilizes annotated data to train and deploy **machine learning models** for **automated species identification** and habitat mapping.

The **scope** of KSO extends to:

- Processing **high-definition** underwater video footage collected from **remotely operated vehicles (ROVs)**.
- Hosting **citizen science campaigns** for **species annotation** via platforms like **Zooniverse**.
- Training and deploying **AI-based detection models** to classify marine species and habitats.
- Providing an **API-based interface** for researchers to interact with trained models and analyze ecological data.

The system is intended for use by **marine biologists, ecologists, AI researchers, and citizen scientists** interested in marine conservation and biodiversity monitoring. Future expansions may include **support for additional marine ecosystems**, **enhanced deep-learning models**, and **improved real-time processing capabilities**.

1.3 System Overview

1.3.1 System Perspective

The **Koster Seafloor Observatory (KSO)** serves as a central hub for managing, processing, and analyzing subsea video data for marine ecological research. It facilitates interactions between **citizen scientists, researchers, storage servers, machine learning models, and biodiversity databases** to streamline underwater biodiversity monitoring.

The system operates through a series of key interactions:

- **Citizen scientists** contribute by creating annotations of marine species in underwater footage.
- **Researchers** upload new footage and contribute to annotation efforts.

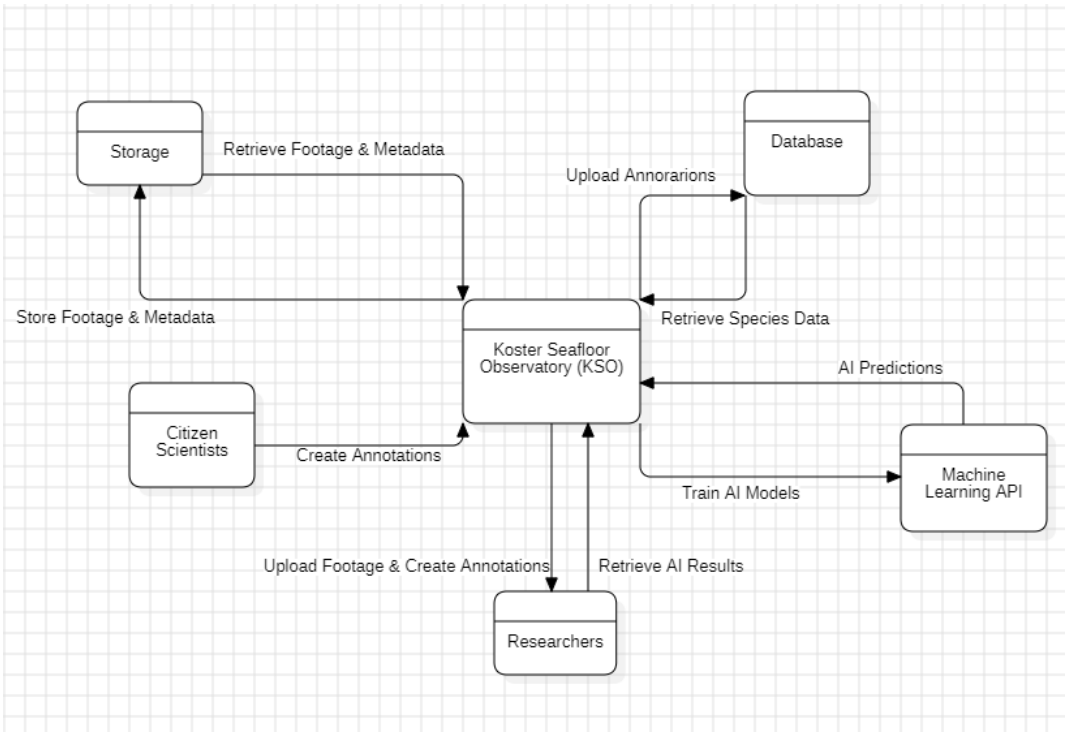

Figure 1.1: System Context Diagram

- **KSO** stores and retrieves footage and metadata from a **storage server** for efficient data management.
- **Machine learning models** are trained using annotations from citizen scientists, and the trained models provide **AI-based species predictions** to researchers.
- **Biodiversity databases** receive and share species data to enhance broader ecological studies.

By integrating **open-source tools, participatory science, and machine learning**, the KSO enables scalable and efficient marine biodiversity monitoring. **Technical implementation details** of these interactions are further explored in **Section 3**.

1.3.1.1 System Interfaces

The KSO system interfaces with several external and internal components:

- **Citizen Science Platform (Zooniverse API)** – Facilitates seamless integration of user annotations into KSO's database.
- **Machine Learning API (FastAPI)** – Provides AI-powered species classification by processing annotated video footage.
- **Storage and Database Services** – Manages metadata, classification data, and video footage in SQL-based storage.
- **Visualization Toolkit** – Offers researchers a graphical interface for exploring and validating ecological data.

1.3.1.2 User Interfaces

KSO provides multiple user interaction points:

- **Web-Based Citizen Science Platform**: A Zooniverse-hosted interface that enables volunteers to annotate species in video clips and images.
- **Researcher Dashboard**: A Streamlit-based front-end for uploading footage, analyzing results, and fine-tuning machine learning model parameters.
- **API-Driven Access**: Advanced users can interact with KSO's trained models and datasets through programmatic API requests.

Future improvements may include an enhanced UI with interactive real-time video annotation tools and improved accessibility for mobile platforms.

1.3.1.3 Hardware Interfaces

The Koster Seafloor Observatory (KSO) operates on a distributed computing architecture that integrates high-performance storage and processing hardware. The key hardware components include:

- **High-Performance Computing (HPC) Server**: Utilized for machine learning training and video processing, equipped with:

  - **GPU**: NVIDIA GTX 2080Ti for accelerated deep learning inference.
  - **CPU**: Intel Core i9-9900 (16 cores, 3.10 GHz).
  - **RAM**: 2GB DDR4 memory.

- **Long-Term Storage (Cold Storage)**: Archive for large video datasets, hosted on university research servers.
- **Short-Term Storage (Hot Storage)**: Linux-based server hosting frequently accessed video files for annotation and model training.
- **Remote Operated Vehicles (ROVs)**: Used for collecting underwater footage; hardware specifications may vary based on deployment.

1.3.1.4 Software Interfaces

KSO integrates various software components for data processing, annotation, and machine learning. The key software interfaces include:

- **Operating System**: Linux-based environment for high-performance computing.
- **Machine Learning Frameworks**: YOLOv3 for training species identification models.
- **Database**: SQLite for metadata storage and annotation tracking.
- **Web Interface**: Developed using Streamlit for researcher access.
- **API Services**:
  - **Zooniverse API**: Handles citizen scientist annotations.
  - **FastAPI**: Facilitates AI model access for automated classification.
  - **Data Visualization API**: Provides mapping and graphical representation of species occurrences.

1.3.1.5 Communication Interfaces

The Koster Seafloor Observatory (KSO) utilizes APIs (FastAPI) for seamless data exchange, a Streamlit web interface for user-friendly model interaction, and Zooniverse for engaging citizen scientists. These interfaces enable efficient communication between data storage, analysis modules, and users, ensuring smooth collaboration in marine ecological research.

1.3.1.6 Memory Constraints

The Koster Seafloor Observatory (KSO) project deals with massive high-resolution video datasets, requiring efficient memory management for storage, processing, and analysis.

Storage constraints arise due to the need to archive large volumes of footage while maintaining a subset in hot storage for active processing. Processing constraints include handling high-resolution video files, extracting frames, and aggregating citizen science annotations, all of which require significant RAM and computational power.

A major limitation is the **2GB DDR4 RAM**, which is **insufficient for such a large-scale project**. Training deep learning models, processing high-resolution imagery, and running real-time inference on large datasets demand significantly more memory. Machine learning models like YOLOv3 require substantial computational resources, and 2GB RAM severely limits performance, causing bottlenecks in data loading, training speed, and inference accuracy.

API and user interaction constraints further strain memory, as handling multiple concurrent requests and visualizing large datasets requires efficient resource allocation. To address these issues, optimization strategies such as data compression, incremental loading, and upgrading to a higher RAM capacity are essential. A more powerful GPU with at least **16GB RAM** is recommended to handle the computational demands of large-scale marine data analysis.

1.3.1.7 Operations

System Operations:

- Store and manage subsea video data
- Process and archive underwater recordings
- Extract frames from video clips
- Manage metadata in a structured database
- Aggregate annotations from citizen scientists
- Train and validate machine learning models
- Run object detection on new video data
- Provide API access for model predictions
- Display machine learning model results
- Authenticate and manage user access

User Operations:

- Upload underwater footage for analysis
- Annotate video clips via citizen science platform
- View and edit species classifications
- Access machine learning predictions via API
- Adjust confidence thresholds for model output
- Visualize spatial distribution of marine species
- Download annotated datasets for research
- Log in to web application for data access
- Customize processing parameters for analysis

1.3.2 System Functions

System functions are defined in Table 1.1

Function	Summary
Data Storage & Retrieval	Provides long-term (cold storage) and short-term (hot storage) data management for efficient archiving and retrieval of subsea footage.
Metadata Management	Organizes movie metadata in a structured SQLite database following Darwin Core (DwC) standards.
Data Processing	Splits long subsea movies into manageable clips and processes annotations for further analysis.
User Engagement	Involves citizen scientists in annotating subsea videos through an intuitive web interface on Zooniverse.
Species Classification	Enables users to classify species present in subsea footage and report their observations.
Object Localization	Allows citizen scientists to mark specific species by drawing bounding boxes on still images.
Data Aggregation	Ensures annotation reliability by applying agreement thresholds before using them in machine learning.
Model Training & Testing	Uses labeled datasets from citizen scientists to train and validate deep learning models (e.g., YOLOv3).
Automated Detection	Deploys trained models to detect and classify species in new subsea footage.
Hyperparameter Tuning	Allows users to adjust detection thresholds (confidence, overlap) to refine model performance.
API Access	Provides an API for researchers to integrate trained models into their own workflows.
Data Visualization	Generates spatial distribution maps of detected species to support ecological research.

Table 1.1: System Functions of Koster Seafloor Observatory (KSO)

1.3.3 Stakeholder Characteristics

There are two main users of the Koster Seafloor Observatory system: researchers and citizen scientists.

- **Researchers** are marine ecologists and other scientific users interested in analyzing subsea footage to study marine biodiversity. They are responsible for uploading underwater footage, using the machine learning models to extract ecological data, and interpreting the results. Researchers need to have a basic understanding of marine species classification, data analysis, and ecological monitoring. They should also be familiar with the system's interface and API to interact with the trained models effectively.

- **Citizen Scientists** are volunteers who contribute by annotating underwater footage on the Zooniverse platform. They help classify species in video clips and mark biological objects in still images. No formal scientific background is required, but they need to follow the provided guidelines and tutorials to ensure accurate annotations. Basic computer skills and the ability to navigate an online annotation platform are necessary for their participation.

1.3.4 Limitations

- **Dependency on Citizen Science Accuracy:** The effectiveness of the software heavily relies on the accuracy of citizen scientist classifications. While an 80% agreement threshold was used to minimize errors, there is still a risk of misclassification, especially for species that are difficult to distinguish from similar taxa.

- **Limited Training Data for Machine Learning:** The machine learning models are trained on a relatively small dataset (409 annotated frames) [2], which may limit their ability to generalize across different underwater environments, lighting conditions, and species. The accuracy of detection models could be improved with larger and more diverse training datasets.

- **Species-Specific Performance:** The current approach has been tested primarily for cold water corals (*Desmophyllum pertusum*). The model's performance may vary when applied to other species, particularly those with less distinct visual features or those that occur in complex backgrounds.

- **Hardware and Computational Requirements:** Running machine learning models and high-performance computing (HPC) tasks requires substantial computational power, including GPUs and high-memory servers. This can limit accessibility for researchers who do not have access to such resources.

- **Storage Constraints:** The software uses a dual-storage approach (long-term and short-term storage), but managing and transferring large volumes of high-resolution video data remains a logistical challenge. Expanding the dataset significantly could lead to storage bottlenecks.

- **Potential Overfitting in Machine Learning Models:** Given the relatively small number of training examples, the model may be prone to overfitting, meaning it could perform well on the training dataset but fail to generalize effectively to new, unseen footage.

- **Limited Real-Time Processing Capabilities:** The system does not currently support real-time analysis of video data. Processing still requires pre-segmentation of long videos into 10-second clips, which may not be ideal for applications requiring immediate feedback, such as live ROV surveys.

- **Limitations of YOLO Algorithm:** Since the machine learning model is based on YOLO (You Only Look Once), it inherits several limitations of this approach:

  - **Struggles with Small Objects:** YOLO may not perform well when detecting small species in underwater imagery, especially if they appear in cluttered or complex backgrounds.

  - **Fixed Grid-Based Detection:** YOLO divides images into a fixed grid,

which may lead to inaccuracies when detecting overlapping or closely positioned species, particularly in dense marine habitats.

- **Sensitivity to Illumination Changes:** The model can struggle with variations in underwater lighting conditions, which can reduce detection performance for species that blend with the background.
- **Bounding Box Precision Limitations:** YOLO predicts bounding boxes at fixed anchor positions, which may result in imprecise localization of irregularly shaped marine organisms.
- **High Confidence Threshold Required for Accuracy:** As observed in the case study, setting an appropriate confidence threshold is crucial for minimizing false positives and false negatives. At lower confidence levels, misclassification rates increase.

1.4 Definitions

- KSO, Koster Seafloor Observatory
- YOLO, You Only Look Once
- API, Application Programming Interface
- AI, Artificial Intelligense
- SQL, Structural Query Language
- ROV, Remote Operated Vehicles
- HPC, High-Performance Computing

2. References

[1] ISO/IEC/IEEE, "ISO/IEC/IEEE International Standard - Systems and Software Engineering – Life Cycle Processes – Requirements Engineering," 2018. ISO/IEC/IEEE 29148:2018(E), pages 1–104.

[2] V. Anton, J. Germishuys, P. Bergström, M. Lindegarth, and M. Obst, "An open-source, citizen science and machine learning approach to analyse subsea movies," *Biodiversity Data Journal*, vol. 9, p. e60548, 2021.

3. Specific Requirements

3.1 External Interfaces

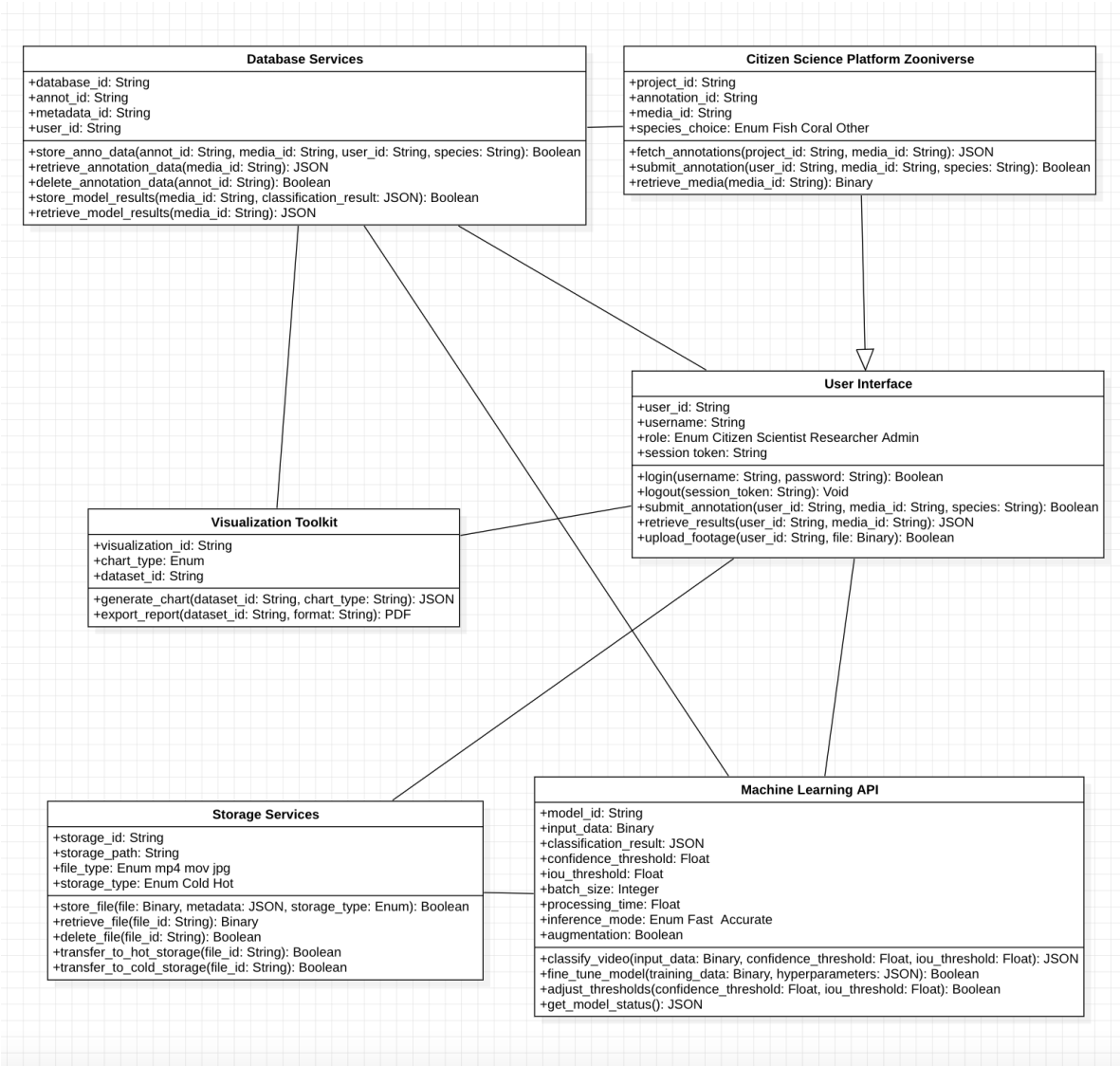

Figure 3.1: External Interfaces Diagram

The **User Interface** allows citizen scientists and researchers to interact with the system via a web platform or API, enabling annotation submission, footage uploads, and result retrieval.

The **Citizen Science Platform** enables volunteers to classify marine species in videos via Zooniverse, aggregating multiple annotations to improve accuracy.

The **Machine Learning API** processes video footage using AI models like TensorFlow and YOLOv3, providing automated species classification with configurable thresholds.

The **Storage Service** manages video and image files, with cold storage for long-term archival and hot storage for frequently accessed data used in annotation and ML processing.

The **Database Service** stores metadata, species classifications, and annotations, ensuring structured data management and interoperability with storage and ML components.

The **Visualization Toolkit** provides interactive graphical tools for researchers to analyze and validate classification results, generate reports, and fine-tune model parameters.

3.2 Functions

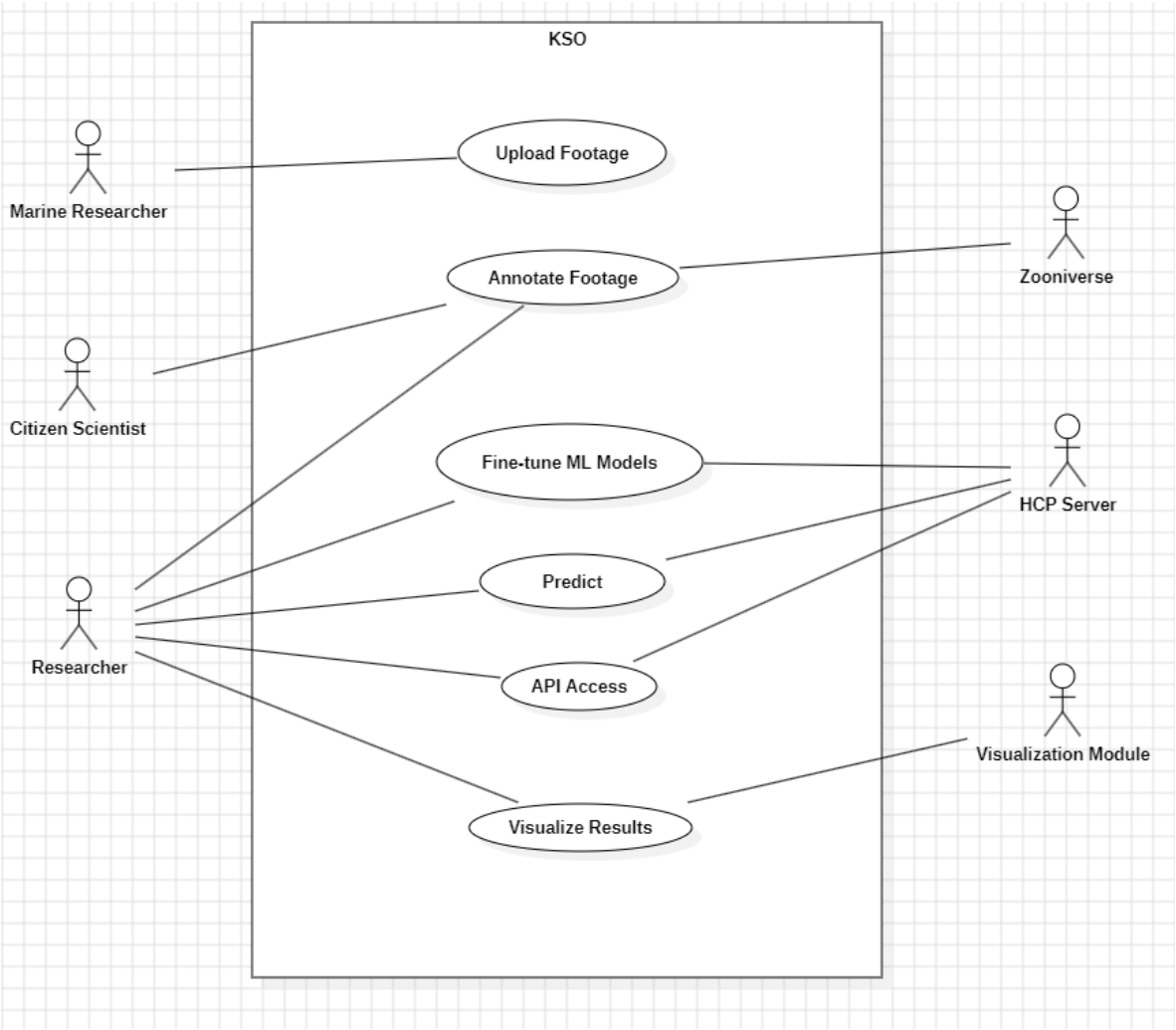

Figure 3.2: Use Case Diagram

Use Case Name	Upload Underwater Footage to KSO
Actors	Marine Researcher
Description	A Marine Researcher uploads long ROV/AUV movie files to the system, which are archived in cold storage and optionally transferred to hot storage for frequent processing.
Pre-Conditions	- Researcher has valid KSO credentials. - Network connection is stable. - Adequate storage is available.
Data	- ROV/AUV video files (1–2 hours each). - Associated metadata (timestamp, location, etc.).
Response	- Files are archived. - Confirmation returned to the researcher. - Metadata recorded in the KSO database.
Stimulus	Researcher initiates the "Upload Footage" action.
Normal Flow	1. Researcher logs into the KSO interface. 2. Researcher selects "Upload Footage" and provides files/metadata. 3. System validates file format and storage availability. 4. System saves files to cold storage and logs metadata. 5. System confirms successful upload to the researcher.
Alternative Flow	If immediate analysis is needed, the system also copies files to hot storage.
Exception Flow	- **File Validation Error:** The system rejects incompatible/corrupted files. - **Insufficient Storage:** The system notifies the researcher and aborts the upload.
Post-Conditions	- Footage and metadata are successfully stored and discoverable in the KSO.
Comments	- May be extended by a metadata-management sub-process if additional edits are required.

Table 3.1: Tabular Description of *Upload Underwater Footage to KSO*

Use Case Name	Annotate Footage via Citizen Science
Actors	Citizen Scientist, Zooniverse, Researcher
Description	Citizen Scientists watch the 10-second clips and identify which species (if any) appear, possibly entering count information and timestamps. The aggregated results are pulled into KSO.
Pre-Conditions	- Clips should have been uploaded to Zooniverse. - Species choices and instructions are configured. - Citizen Scientist has access to the project.
Data	- 10-second video clips. - Species/label lists.
Response	- Annotation responses recorded in Zooniverse. - Each clip is retired once enough classifications are gathered.
Stimulus	Citizen Scientist visits the Koster Seafloor Observatory project in Zooniverse and requests a new clip to classify.
Normal Flow	1. Citizen Scientist logs in (or continues anonymously, if allowed). 2. Zooniverse serves a 10-second clip. 3. Citizen Scientist selects species, counts, and timestamps. 4. Classification is submitted and stored. 5. Once the clip meets the required number of classifications, it is retired. 6. Aggregated data is periodically transferred to KSO.
Alternative Flow	- Multiple species can be selected per clip. - "Unsure" or "Nothing here" can be chosen for ambiguous clips.
Exception Flow	- **Platform Unavailable:** Classification cannot proceed. - **Network Timeout:** Submitted data may be lost if the session disconnects.
Post-Conditions	- Citizen Scientist input is used for consensus labeling in KSO's data module.
Comments	- This includes or precedes the aggregation of classifications (see Use Case 4).

Table 3.2: Tabular Description of *Annotate Footage via Citizen Science*

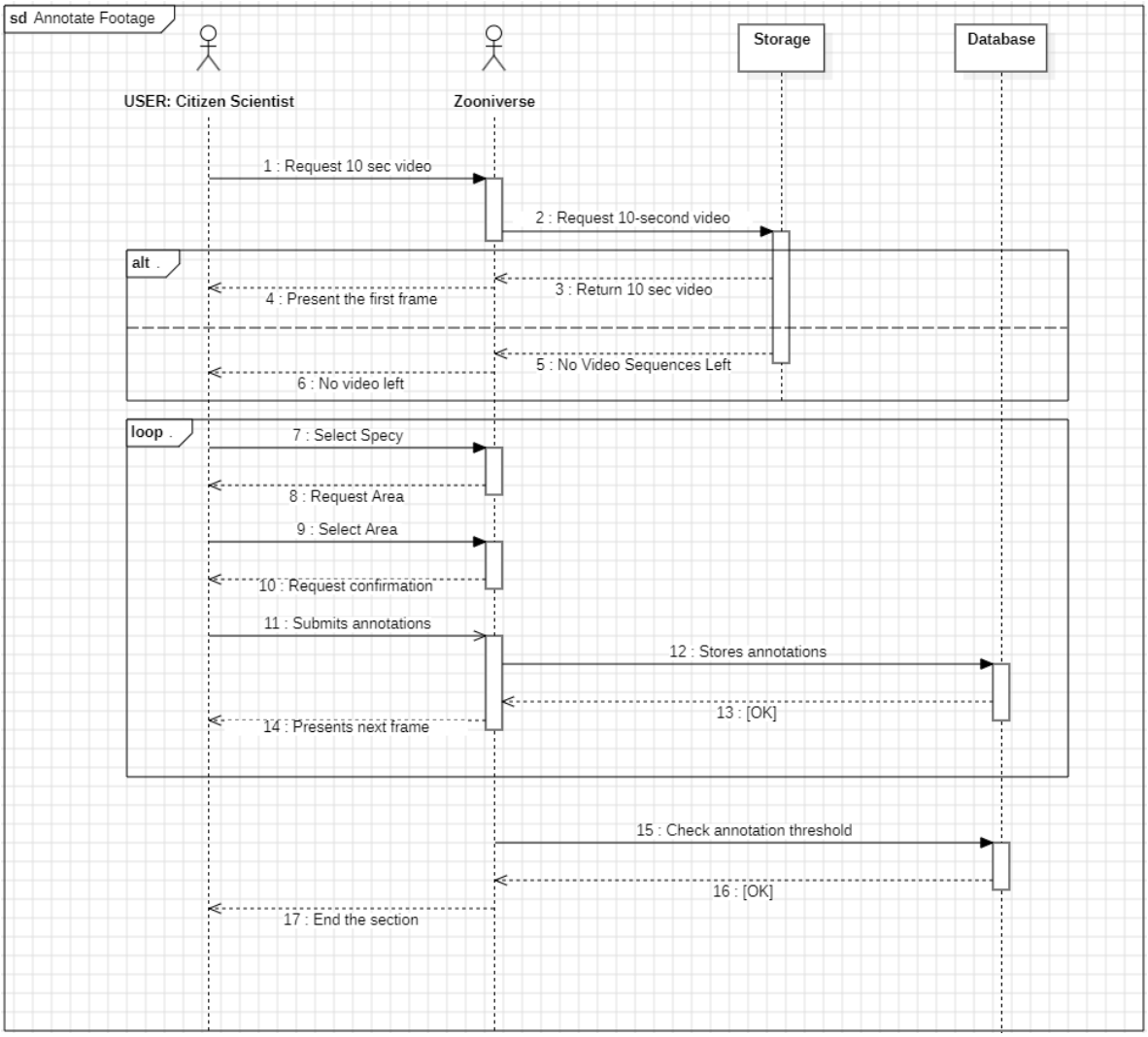

Figure 3.3: Sequence Diagram for Annotate Footage

Use Case Name	Fine-tune Machine Learning Models
Actors	Researcher, HPC Server
Description	Fine-tune machine learning models using citizen scientist-annotated data for automated species detection in subsea footage.
Pre-Conditions	- Annotated datasets ready and validated; machine learning infrastructure available.
Data	- Annotated video/image datasets - Data augmentation scripts preprocessing tools - Machine learning scripts
Response	- Machine learning model fine-tuned, validated, and available for further application.
Stimulus	Researcher initiates the ML model fine-tuning process.
Normal Flow	1. The Researcher uploads the annotated subsea imagery dataset along with fine-tuning parameters (epochs, learning rate, early stopping, confidence thresholds). 2. The KSO API validates the dataset and parameter integrity, confirming they meet the required standards. 3. The HPC Server iteratively trains and validates the machine learning model, monitoring performance metrics until the defined epochs are completed or early stopping criteria are met. 4. Upon completion, the HPC Server evaluates performance metrics (F1-score, precision, recall, mAP) and securely stores the trained model. 5. The Researcher reviews the fine-tuning outcomes and either accepts the results or revisits parameters for further iterations, returning to step 2 if necessary.
Alternative Flow	- Researcher modifies hyperparameters and re-tunes the model based on initial evaluation metrics.
Exception Flow	- If computational resources are insufficient or there are script errors, training halts, and the issue is logged and reported to the system administrator.
Post-Conditions	- Machine learning model is accurately tuned, validated, and ready to predict ecological observations from new footage.
Comments	- Ensuring data quality and appropriate parameter tuning are critical for the accuracy and reliability of the tuned model.

Table 3.3: Tabular Description of *Fine-tune Machine Learning Models*

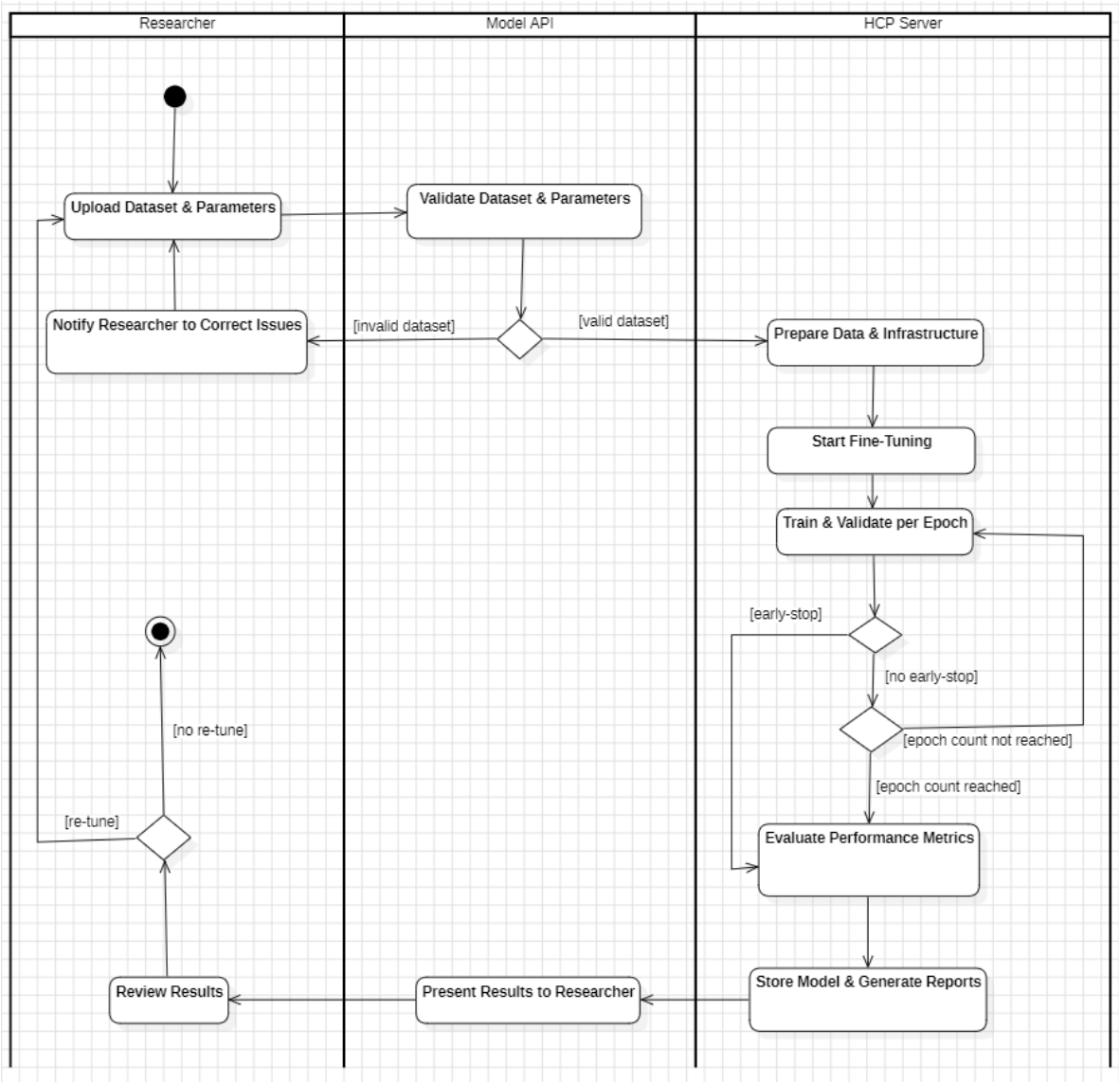

Figure 3.4: Activity Diagram of Fine-tuning ML Models

Use Case Name	Running Predictions on New Footage
Actors	Researcher, HPC Server
Description	The trained model is used to detect target species in newly uploaded or unseen footage, generating bounding boxes and presence/absence data.
Pre-Conditions	- A trained model is available. - Inference thresholds (confidence, IOU) are set.
Data	- Model weights. - New video/image frames. - Inference parameters.
Response	- Detected bounding boxes with confidence scores. - Presence/absence data logged in the KSO database.
Stimulus	Researcher selects "Analyze New Footage" or calls the ML API.
Normal Flow	1. Researcher chooses which footage to process. 2. System loads the model and HPC resources. 3. Model runs inference per frame/clip, generating bounding boxes. 4. Consecutive bounding boxes are combined if necessary (e.g., continuous detection). 5. Results are returned to the researcher.
Alternative Flow	Researcher adjusts confidence or IOU thresholds, re-runs inference to fine-tune detection results.
Exception Flow	- **Model Version Missing:** The system prompts user for a valid model. - **HPC Failure:** Job is interrupted; system logs error and notifies researcher.
Post-Conditions	- Updated detection data is stored for subsequent mapping or time-series analysis. - The KSO can render distribution maps or other summaries.
Comments	- Often extended by a "Visualize and Report Biodiversity Data" use case for output data.

Table 3.4: Tabular Description of *Running Predictions on New Footage*

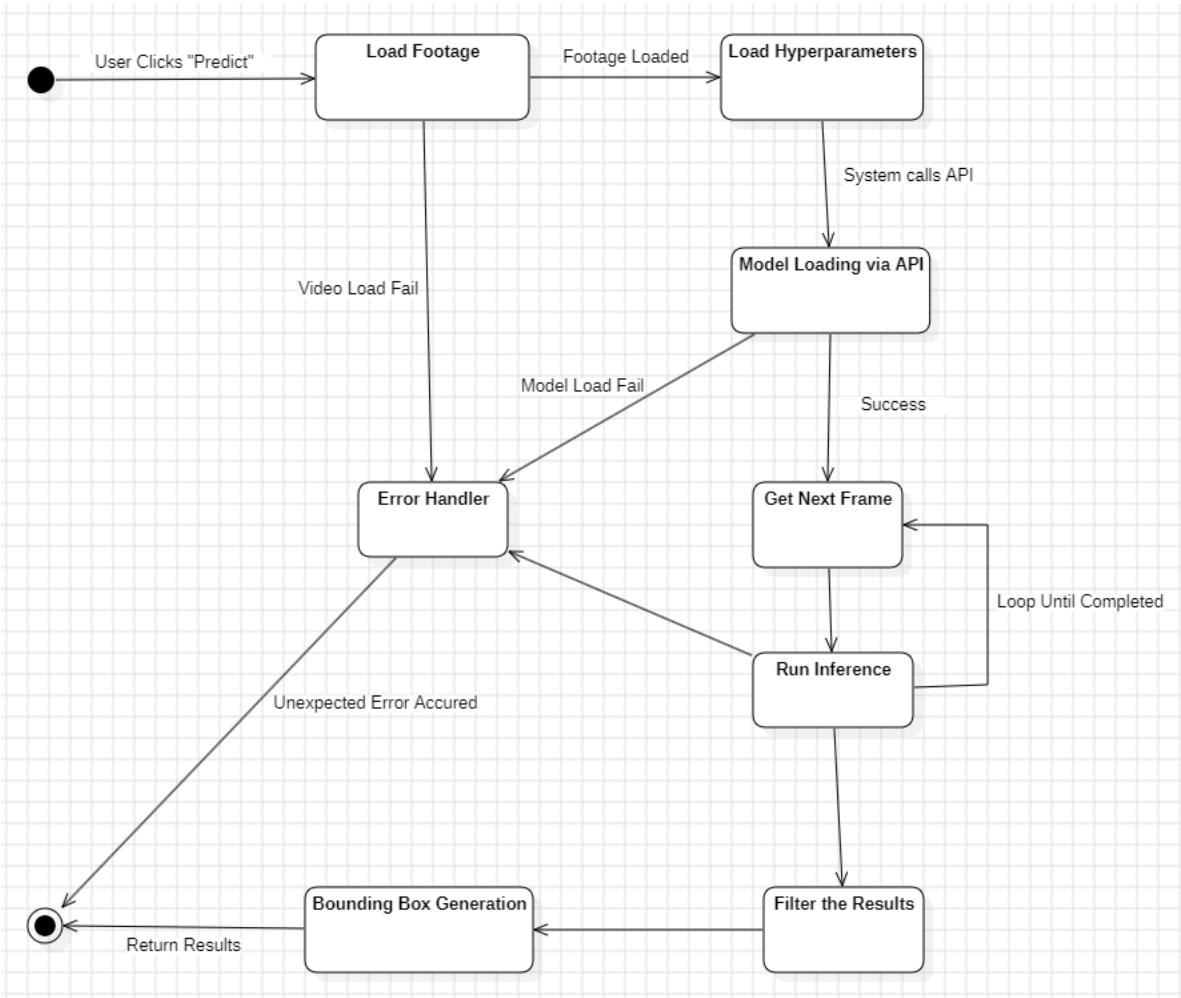

Figure 3.5: State Diagram of Running Predictions

Use Case Name	Visualize and Report Biodiversity Data
Actors	Researcher, Visualization Module
Description	Visualizes biodiversity data spatially and temporally, providing interactive maps and summaries for monitoring and reporting purposes.
Pre-Conditions	- Annotations from ML model and citizen scientists are available. - Biodiversity data stored and accessible in KSO database.
Data	- Model-generated annotations. - Citizen scientist annotations. - Spatial and temporal metadata.
Response	- Interactive maps showing species occurrences. - Time-series graphs. - Downloadable reports and summaries.
Stimulus	Researcher or manager selects "Visualize Data" option in the system.
Normal Flow	1. User specifies area or time period. 2. System retrieves relevant biodiversity data from the database. 3. System generates visualizations (maps, graphs). 4. User reviews visualizations and optionally generates reports.
Alternative Flow	User customizes visualization parameters or exports data for external analysis.
Exception Flow	**- Data Retrieval Error:** System alerts the user and requests revised parameters.
Post-Conditions	- Visualizations or reports are saved for future reference. - Generated insights used for conservation management decisions.
Comments	- Often integrated with export functions for detailed reporting.

Table 3.5: Tabular Description of *Visualize and Report Biodiversity Data*

Use Case Name	API Access for External Researchers
Actors	Researcher, HCP Server
Description	External researchers access KSO's trained models via API.
Pre-Conditions	- API credentials are issued to external users. - API is deployed and running on FastAPI.
Data	- Request parameters (video URL, confidence threshold) - Detection results.
Response	- JSON response with species detection results.
Stimulus	External researcher submits an API request.
Normal Flow	1. User specifies area or time period. 2. System retrieves relevant biodiversity data from the database. 3. System generates visualizations (maps, graphs). 4. System generates visualizations (maps, graphs). 5. Researcher processes the results for further analysis.
Alternative Flow	- Researcher can adjust API parameters (IoU, confidence threshold).
Exception Flow	- If API is overloaded, request is queued.
Post-Conditions	- Researcher obtains automated species detection data.
Comments	- Enables broader collaboration and integration into research projects.

Table 3.6: Tabular Description of *API Access for External Researchers*

3.3 Logical Database Requirements

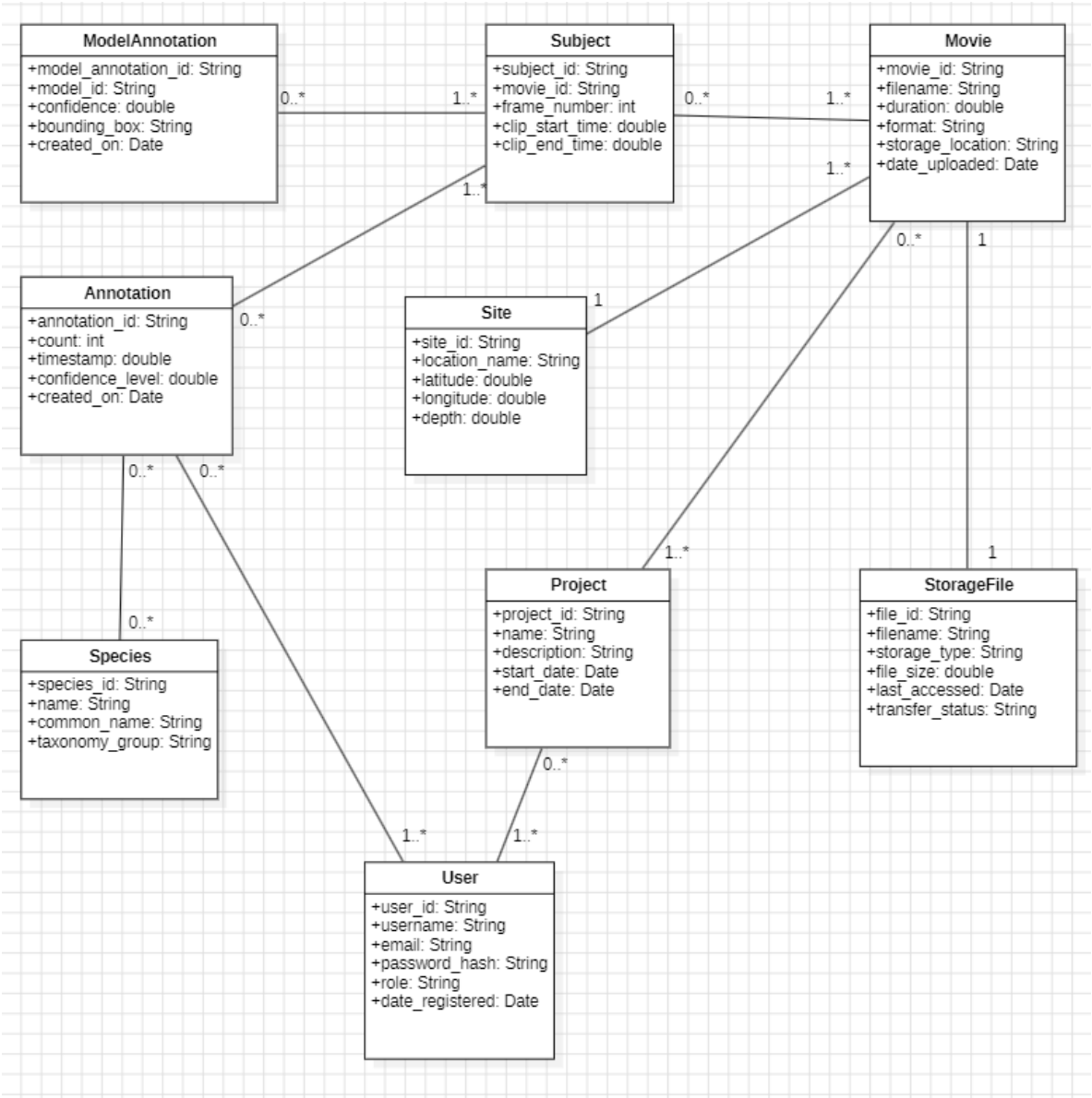

Figure 3.6: Logical Database Diagram

The logical database for the Koster Seafloor Observatory (KSO) efficiently manages user annotations, AI classifications, and ecological video data. It consists of interrelated tables that store movies, species, annotations, and storage files, ensuring struc-

tured data management. Users participate in projects, contributing to species classification on video clips (subjects), with AI models further refining these classifications. The database supports 1-to-1, 1-to-Many, and Many-to-Many relationships for efficient querying and retrieval. It enables seamless integration between citizen science efforts, machine learning automation, and marine research workflows.

3.4 System Quality Attributes

Dependability properties, Maintainability, and so on) in the order of priority with associated requirements. Usability, Performance and at least one additional quality attribute must be considered.

3.4.1 Usability Requirements

- The system should be easy to install, configure, and operate, ensuring accessibility for researchers, citizen scientists, and other users with minimal technical expertise.
- The KSO platform should provide an intuitive and user-friendly interface for both data annotation and analysis, allowing seamless participation from citizen scientists.
- The system should support interactive data visualization tools that simplify the understanding and interpretation of marine ecological data.
- The interface should be accessible, incorporating features such as screen reader compatibility and adjustable font sizes to accommodate different user needs.
- The platform should automate repetitive tasks such as data pre-processing, annotation aggregation, and model training, reducing the manual workload for researchers.
- The system should provide real-time feedback and clear status indicators for data uploads, annotation progress, and model performance.

- The web interface should be aesthetically pleasing, engaging, and designed to enhance user motivation, especially for citizen scientists contributing to the classification process.

3.4.2 Performance Requirements

- The system must support efficient handling and processing of large video datasets, ensuring smooth playback and frame extraction.
- Data transfer between storage, processing modules, and machine learning components should be optimized for minimal latency.
- The platform should leverage high-performance computing (HPC) resources to efficiently train machine learning models on large annotated datasets.
- The machine learning API should process requests in real-time, allowing researchers to obtain species identification results quickly.
- The system should handle concurrent user interactions, ensuring smooth annotation workflows on the citizen science platform.

3.4.3 Reliability Requirements

- The system should ensure accurate and lossless transfer of video, annotation, and model prediction data between different modules.
- The machine learning model should produce consistent and reproducible results with a high confidence threshold (e.g., 0.7) to minimize errors in species identification.
- The database must maintain data integrity, ensuring that annotations from citizen scientists and model outputs are correctly stored and retrievable.

3.4.4 Maintainability Requirements

- The system should follow a modular architecture, allowing independent updates and improvements to data management, citizen science, and machine learning components.
- Comprehensive documentation should be provided to facilitate future development, debugging, and system expansion.
- Version control systems (e.g., Git) must be used to track changes and enable rollbacks if necessary.
- Automated unit tests and integration tests should be developed to identify and resolve issues efficiently.

3.4.5 Portability Requirements

- The KSO web platform should follow modern web standards to ensure compatibility with different devices and browsers.
- The machine learning models should be deployable across various computational environments, including cloud-based and on-premise HPC systems.
- The citizen science platform should be accessible from both desktop and mobile devices to maximize user engagement.

3.5 Design Constraints

The design of the Koster Seafloor Observatory is subject to the following constraints imposed by external factors, including regulatory requirements, official standards, and organizational limitations:

- **Compliance with Data Standards**

  - The system follows the *Darwin Core (DwC) Standards* [2] to ensure compatibility and interoperability with global biodiversity data repositories.
  - Metadata associated with the underwater footage is regularly published in the *Swedish National Data Archive*, requiring adherence to data formatting and archival standards.

- **Open-Source Software Requirements**
  - The system is built entirely on *open-source technologies*, which mandates that all software components, including machine learning models and citizen science tools, be freely available for modification and redistribution.
  - The observatory must align with the principles of *open science* and ensure that the tools and datasets can be accessed and reused by the broader research community.

- **Regulatory and Ethical Compliance in Citizen Science**
  - The use of *Zooniverse* for citizen science participation requires compliance with *ethical guidelines* related to data privacy, user engagement, and volunteer contributions.
  - Data collection involving human participants must adhere to the *General Data Protection Regulation (GDPR)*, ensuring that personal data is handled securely and that no sensitive user information is collected.

- **Hardware and Computational Constraints**
  - The system must operate within the specifications of the *High Performance Computing (HPC) server*, which includes a *GTX2080Ti GPU, Intel Core i9-9900 CPU, and 2GB DDR4 RAM* [2].
  - Machine learning algorithms must be optimized to function within these hardware limitations, balancing computational efficiency with model accuracy.

- **Legal and Institutional Data Sharing Policies**
  - Data stored and processed by the observatory must comply with *Swedish national regulations* on research data management and *European Union directives on marine environmental protection.*
  - Collaboration with institutions such as *the University of Gothenburg and Chalmers University of Technology* requires alignment with their internal data storage, sharing, and security policies.

3.6 Supporting Information

The **Koster Seafloor Observatory** is an **open-source** project designed to facilitate large-scale marine ecological research. The source code is publicly available on **GitHub**, allowing researchers, developers, and citizen scientists to explore and contribute to the project. Additionally, there are numerous open issues on the repository that require solutions, offering opportunities for contributors to support the project by developing and optimizing various modules.

Apart from contributing as a developer, individuals can participate in the project as **data annotators** through the **Zooniverse citizen science platform**. Volunteers can help classify marine species from underwater footage, improving the training dataset for machine learning models. Furthermore, researchers can validate machine learning outputs and provide expert classifications to enhance model accuracy.

The **Koster Seafloor Observatory** also offers an **Application Programming Interface (API)** and a **user-friendly web interface** for researchers to test machine learning models on new underwater footage. The system ensures compliance with **data-sharing standards**, including **Darwin Core (DwC)**, and follows **General Data Protection Regulation (GDPR)** guidelines to protect user privacy.

This supporting information is **not considered part of the core software requirements** but serves as valuable background material for users, contributors, and researchers engaging with the system.

4. Suggestions to Improve The Existing System

4.1 System Perspective

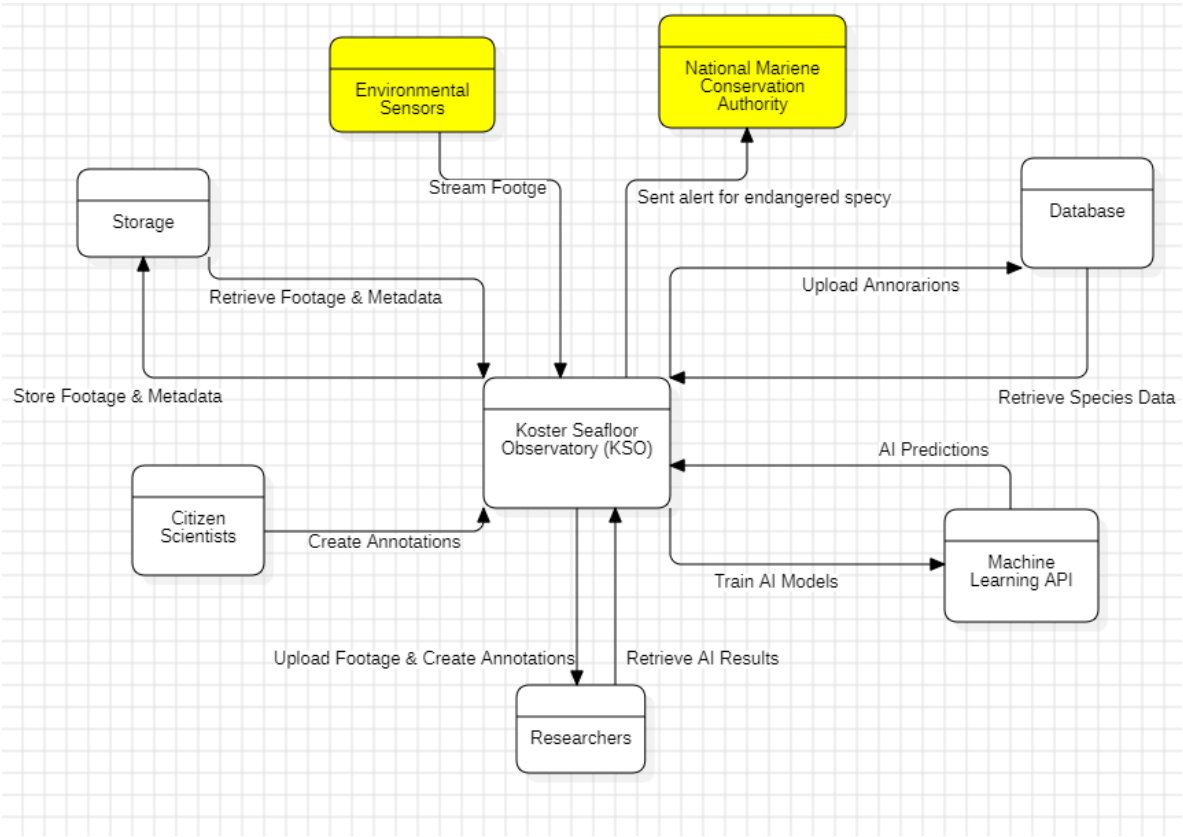

Figure 4.1: Improved System Context Diagram

Environmental Sensors (Video Stream for Live Detection): Environmental sensors integrated within the KSO platform provide continuous subsea video streams, facilitating real-time detection and identification of marine species via AI-driven models. This integration significantly enhances biodiversity assessments and supports immediate ecological analysis and decision-making.

National Marine Conservation Authority (Alerts for Extinct Species): The KSO system automatically issues instantaneous alerts to the National Marine Conservation Authority upon the detection of extinct or critically endangered marine species. This proactive alert mechanism ensures rapid conservation responses and timely protection measures for vulnerable marine biodiversity.

4.2 External Interfaces

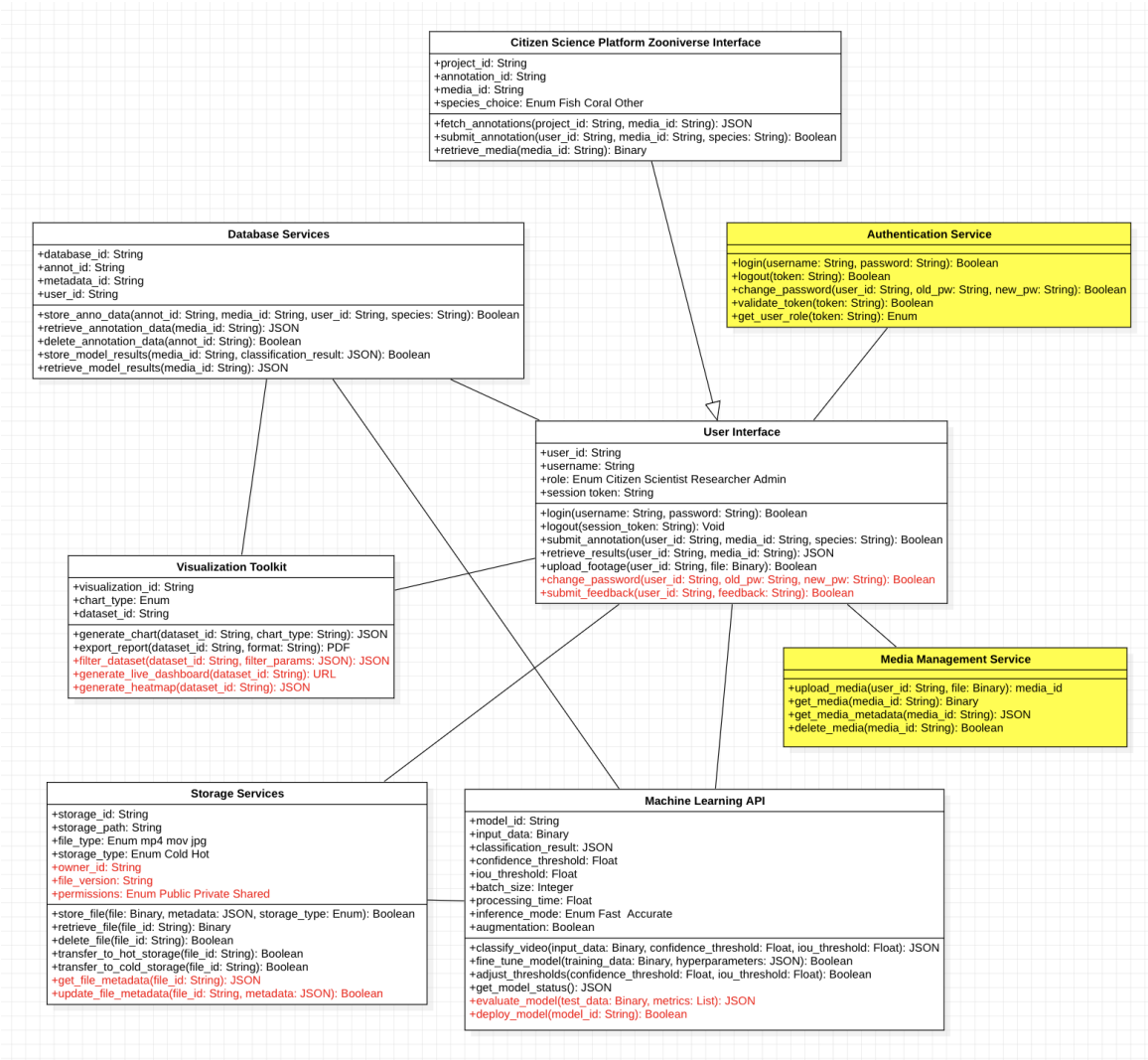

Figure 4.2: Improved External Interfaces Diagram

The **Authentication Service** has been added in order to isolate the login, logout, and token validation logic for users from the **User Interface** class. It is an example of the use of the principle of separation of concerns and makes authentication safe for reuse by other components like APIs and background services. Centralization of session handling, role authentication, and token verification makes the system scalable, testable, and maintainable.

The **Media Management Service** was introduced to centralize media operations such as uploading, retrieval, deletion, and accessing metadata. Media management in the current design is scattered between the **User Interface** and **Zooniverse** classes, leading to duplication and unbalanced logic. A standalone media manager offers better control, clearer responsibilities, and flexibility to deal with different file types, metadata, and storage levels across the system.

The **Machine Learning API** was also supplemented with model evaluation and deployment. These are genuine requirements in practical ML systems in which models have to be retrained, tested, monitored, and deployed in incremental steps. With fine-tuning, deployment, and monitoring of status enabled, the API is an end-to-end ML lifecycle manager enabling continuous improvement and good automation.

The **Visualization Toolkit** was enhanced to support filtering, real-time dashboards, heatmap generation, and export in multiple formats. These improvements empower users and researchers to extract more meaningful insights from their data and reports. By enabling dynamic interaction with datasets, the toolkit becomes a powerful interface for data exploration, which is especially valuable in citizen science and research-oriented platforms like KSO.

Storage Services were also improved to enable metadata access, file versioning, permissions, and user-based file listings. The upgrade enables support for handling higher volumes of data more effectively with fine-grained control over file ownership, visibility, and lifecycle. These features are aligned with cloud storage best practices and are required to offer secure, multi-user environments.

4.3 Functions

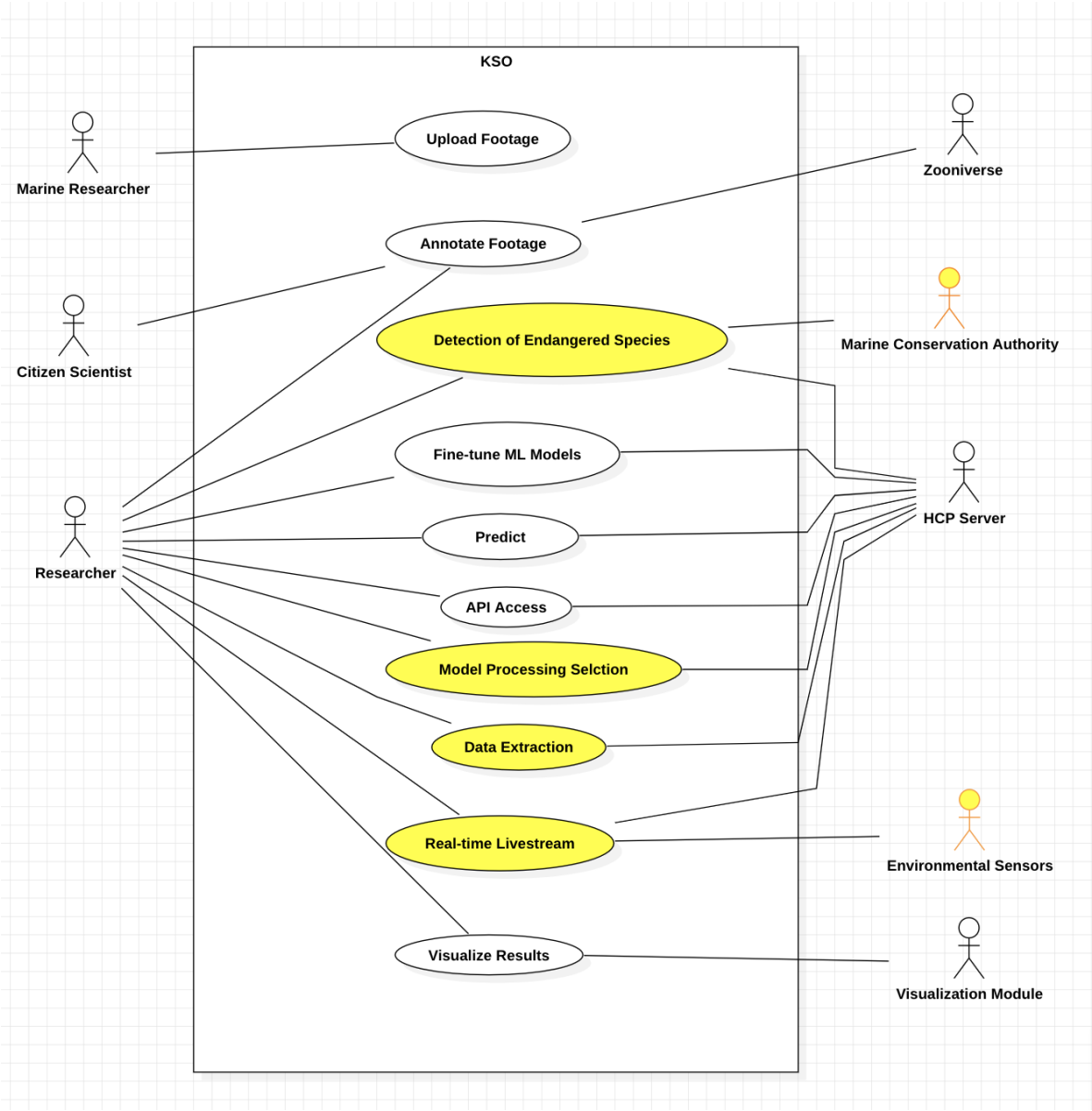

Figure 4.3: Improved UseCase Diagram

Use Case Name	Detection of Endangered or Possibly Extinct Marine Species
Actors	HPC Server, Researcher, Marine Conservation Authority
Description	The system detects rare species in multiple frames and alerts researchers and conservation authorities.
Pre-Conditions	- Species detected in multiple frames. - Confidence score should be higher than 90%. - Species exists in endangered/extinct sea creatures database. - Conservation authority contact available.
Data	- Species name, confidence score, coordinates, timestamp, video frame. - Historical records (if available).
Response	- System flags detection and notifies researcher. - If verified, an alert is sent to the authority.
Stimulus	ML model detects an endangered or extinct species across multiple frames.
Normal Flow	1. System processes video and detects species. 2. Checks confidence level and number of frames. 3. Cross-references with species database. 4. If valid, flags detection and notifies researcher. 5. Researcher verifies detection. 6. If confirmed, system alerts conservation authority.
Alternative Flow	If researcher finds an error, detection is discarded and model is adjusted.
Exception Flow	- If confidence is lower than 90%, system requests further verification. - If no authority response, system retries later.
Post-Conditions	- Verified sightings logged in biodiversity records. - Data supports conservation actions.
Comments	- Ensures high accuracy by requiring multiple frame detections. - Supports real-time conservation efforts.

Table 4.1: Tabular Description of *Detection of Endangered or Possibly Extinct Marine Species*

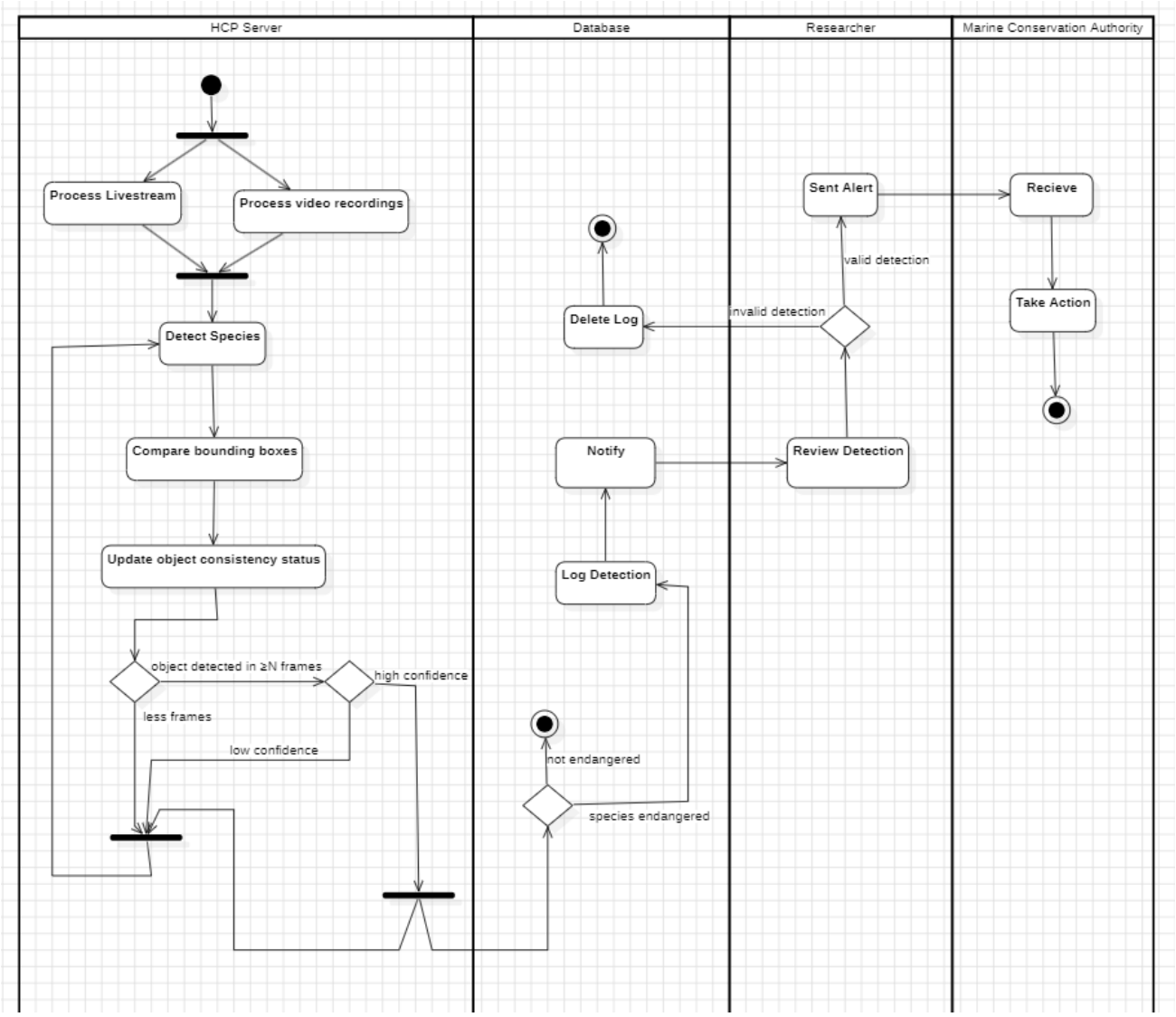

Figure 4.4: Activity Diagram of Detection of Endangered or Possibly Extinct Marine Species

Use Case Name	Real-Time Marine Monitoring via Livestream and Environmental Sensors
Actors	Environmental Sensors, Researcher, HPC Server
Description	The system processes livestream footage and environmental data to monitor marine life in real time.
Pre-Conditions	- Livestream cameras and environmental sensors are active. - AI model processes incoming frames. - Internet connectivity is stable.
Data	- Video stream data, detected species, confidence score. - Environmental parameters (temperature, pH, oxygen levels, salinity).
Response	- System displays detected species in real-time. - Researchers receive notifications of notable detections. - Environmental changes are logged and analyzed.
Stimulus	The system receives continuous livestream and sensor data.
Normal Flow	1. System continuously processes livestream footage. 2. AI detects marine species and classifies them in real time. 3. System overlays detection data on the live feed. 4. Environmental sensor data is correlated with detected species. 5. Users view real-time species identification through an interactive dashboard. 6. System logs detections and environmental conditions for further analysis.
Alternative Flow	- If AI confidence is low, system requests additional frames before confirming detection. - If no species is detected, system continues monitoring without notifications.
Exception Flow	- If livestream connectivity fails, system retries connection. - If environmental sensors provide anomalous readings, system alerts researchers.
Post-Conditions	- Detected species and environmental data are logged. - Users and researchers gain real-time insights into marine biodiversity.
Comments	- Enables real-time marine life monitoring for research and conservation. - Can be integrated with public educational platforms for citizen science.

Table 4.2: Tabular Description of *Real-Time Marine Monitoring via Livestream and Environmental Sensors*

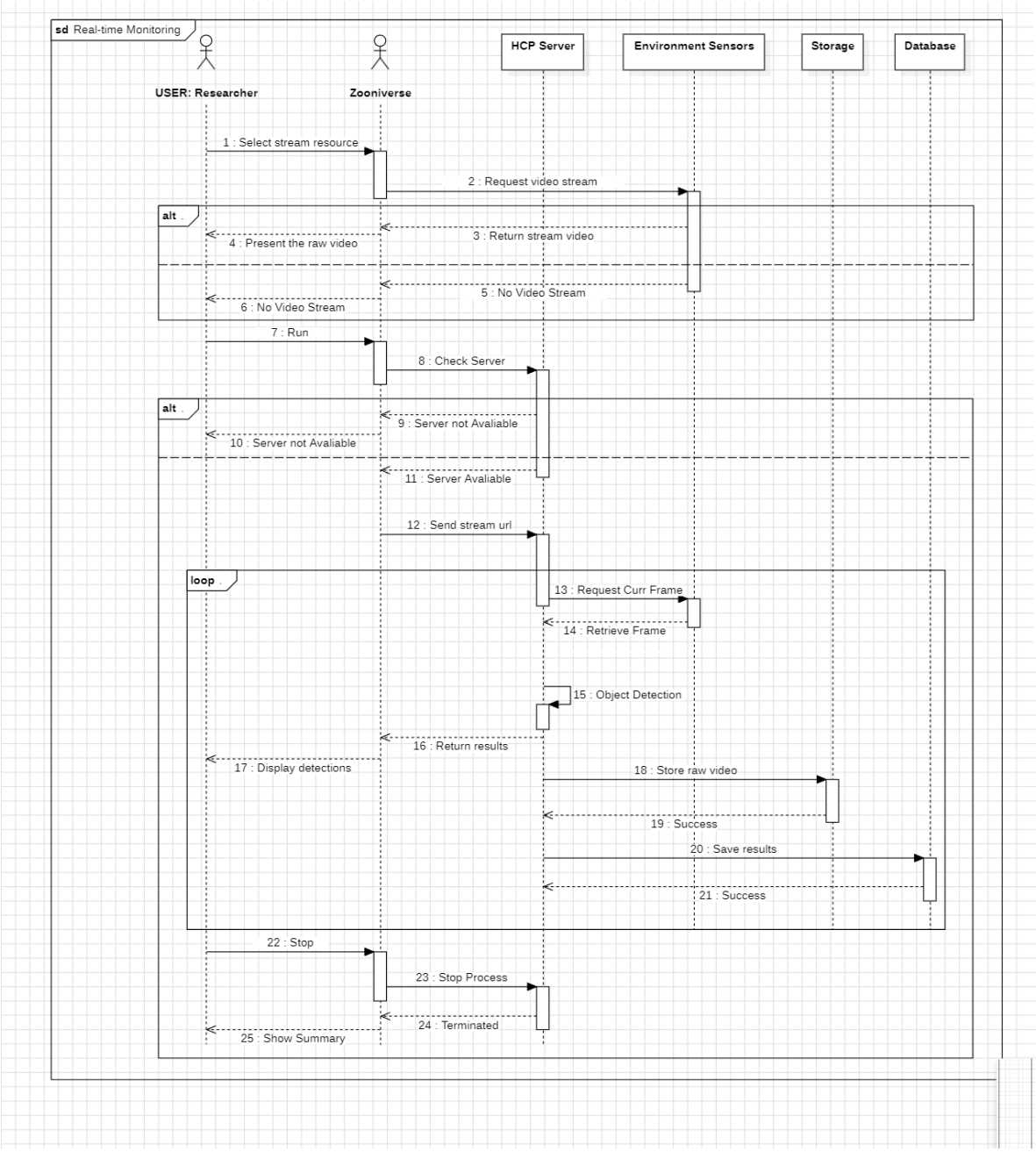

Figure 4.5: Sequence Diagram of Real-Time Marine Monitoring via Livestream and Environmental Sensors

Use Case Name	Model Processing Selection: GPU vs. TPU
Actors	Researcher, HCP Server
Description	Users can choose between GPU and TPU for model execution, with TPU usage limited by available credits.
Pre-Conditions	- The system has access to both GPU and TPU resources. - TPU credits are available. - User has the appropriate permissions to select hardware.
Data	- User's hardware selection (GPU/TPU). - Remaining TPU credits. - Model processing time and resource usage logs.
Response	- The system runs the model on the selected hardware. - TPU credits are deducted if TPU is used. - System notifies the user if credits are insufficient.
Stimulus	User selects model execution hardware (GPU or TPU).
Normal Flow	1. User accesses model execution settings. 2. User selects GPU (default) or TPU for processing. 3. System checks TPU credit balance. 4. If credits are sufficient, TPU execution starts, and credits are deducted. 5. If GPU is selected, the model runs without restrictions. 6. System processes the model and returns results. 7. Usage is logged for performance tracking.
Alternative Flow	- If TPU credits are low, system suggests switching to GPU or allows the user to purchase additional credits. - If TPU is at capacity, system queues the job or offers a GPU alternative.
Exception Flow	- If TPU execution fails, the system falls back to GPU processing. - If the GPU is overloaded, the system suggests scheduling the job later.
Post-Conditions	- Model execution completes on the selected hardware. - TPU credits are updated if used. - Execution logs are stored for tracking and optimization.
Comments	- Similar to Google Colab, allowing flexible TPU/GPU selection. - Ensures efficient resource allocation based on availability and user needs.

Table 4.3: Tabular Description of *Model Processing Selection: GPU vs. TPU*

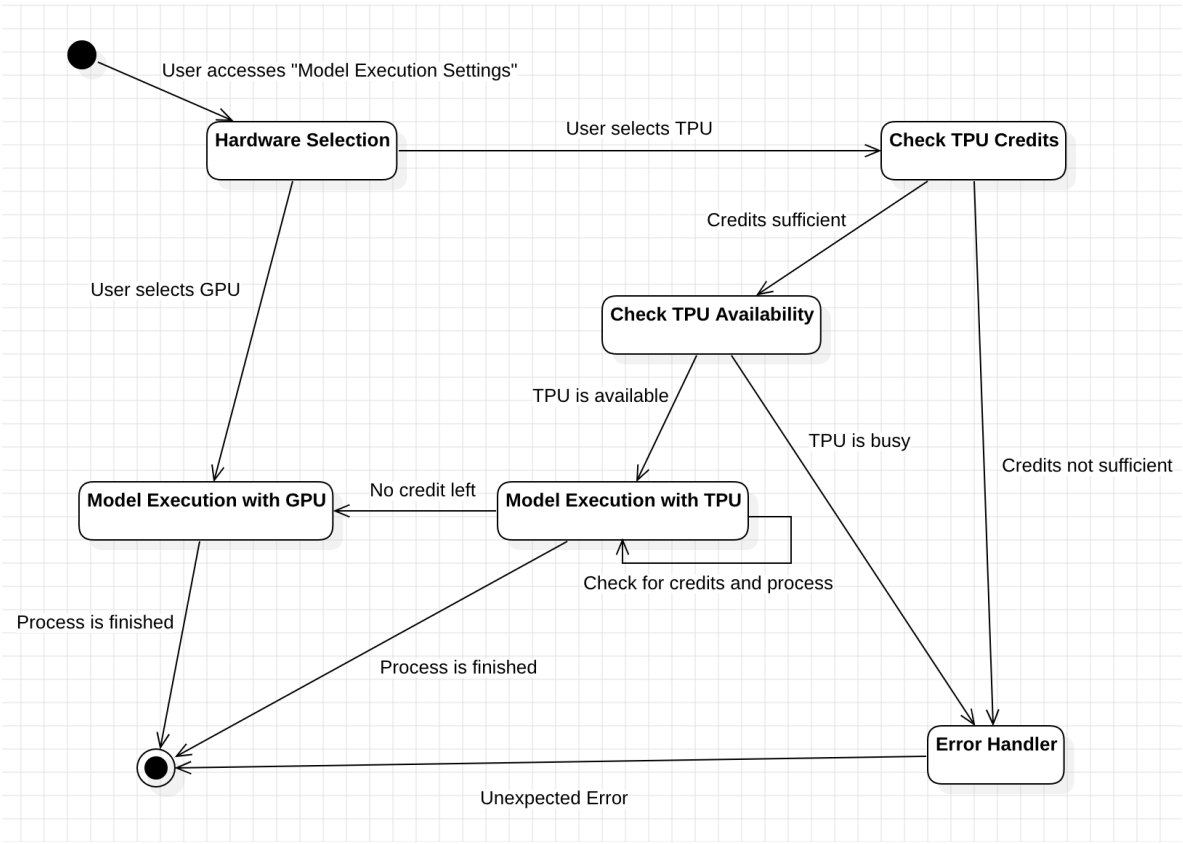

Figure 4.6: State Diagram of Model Processing Selection: GPU vs. TPU

Use Case Name	Data Extraction for Researchers
Actors	HPC Server, Researcher
Description	Researchers can extract processed biodiversity and environmental data for analysis and reporting.
Pre-Conditions	- The researcher has access to the system. - Required data (species detections, environmental readings) is available in the database. - User has permissions for data export.
Data	- Species detection records (location, time, confidence score). - Environmental sensor data (temperature, pH, oxygen levels, etc.). - Model predictions and annotations. - Export format selection (CSV, JSON, API).
Response	- HPC Server generates and provides the requested dataset. - Large datasets may be compressed before download. - API users receive structured data responses.
Stimulus	Researcher requests a data extraction.
Normal Flow	1. Researcher logs into the system. 2. Researcher selects desired data type and time range. 3. HPC Server retrieves requested records from the database. 4. Researcher chooses export format (CSV, JSON, API response). 5. HPC Server processes the extraction request. 6. Data is provided for download or accessed via API. 7. Extraction log is recorded.
Alternative Flow	- If dataset is large, system notifies user about processing time. - If multiple formats are needed, system allows batch export.
Exception Flow	- If no matching data is found, system notifies researcher. - If database is under maintenance, system queues the request.
Post-Conditions	- Extracted data is available for further research. - System logs the extraction for tracking purposes.
Comments	- Supports biodiversity research and marine conservation analysis. - API access allows integration with external research tools.

Table 4.4: Tabular Description of *Data Extraction for Researchers*

4.4 Logical Database Requirements

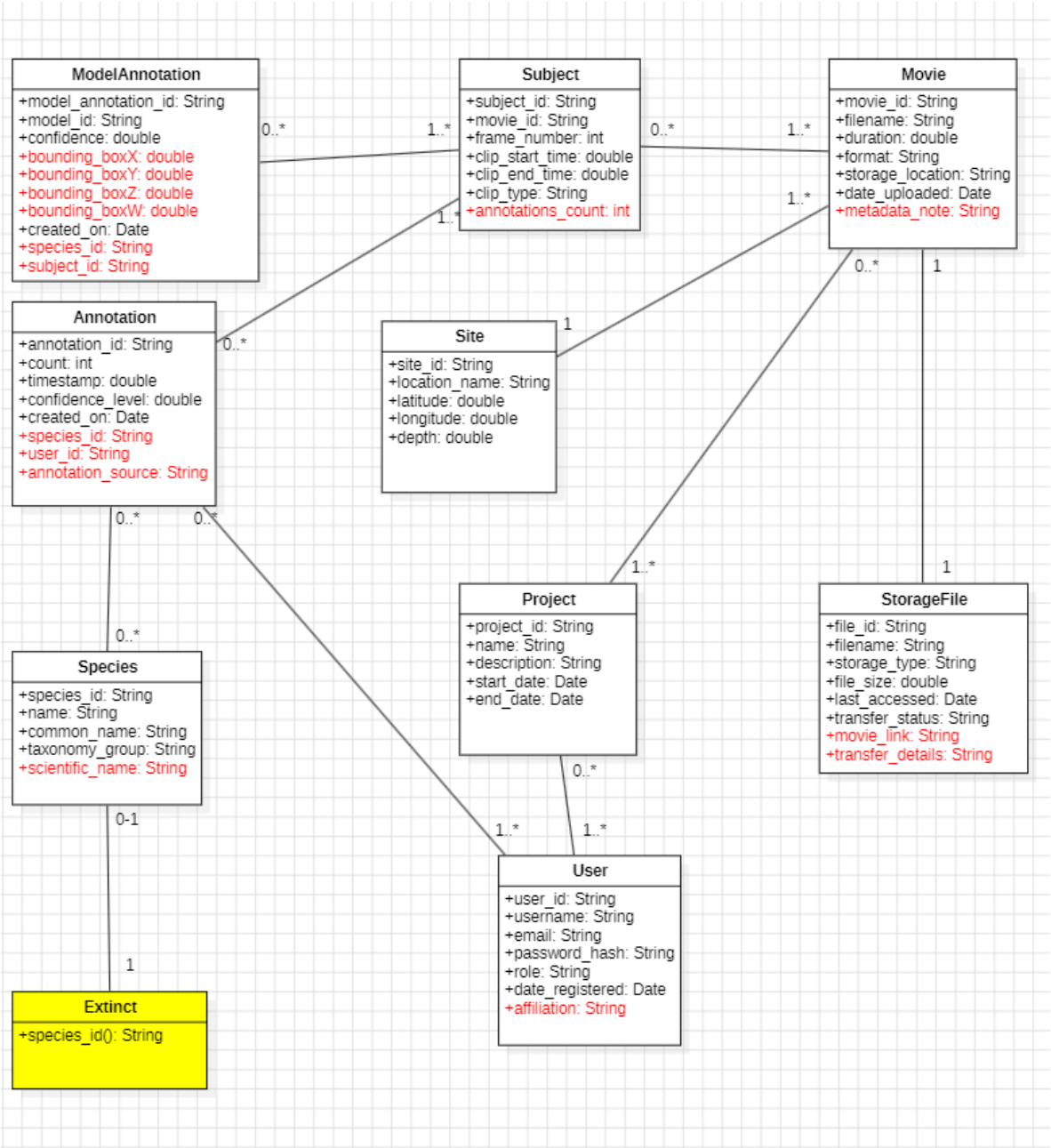

Figure 4.7: Improved Logical Database Diagram

We introduced several new fields to enhance clarity and flexibility: `species_id` and `scientific_name` provide consistent references for each annotated organism, while `user_id` and `annotation_source` distinguish who or what created each annotation (e.g. expert, citizen scientist, or AI). By adding `subject_id`, we ensure annotations align precisely with specific clips or frames, and `bounding_box_coordinates` captures the exact location of the detected species within an image. To store additional context, we included `metadata_note` in our media records, and for user profiles, we added `affiliation` to track organizational or institutional ties. Additionally, to support monitoring and rapid conservation alerts, we introduced a new table, `Extinct`, which references the `species_id` as a foreign key, clearly identifying species marked as extinct or critically endangered.

4.5 System Quality Attributes

4.5.1 Suggestions to Improve Usability Requirements

- **Guided Tutorials:** Integrate short, interactive video or text-based tutorials tailored to first-time users, helping them understand annotation protocols and platform features more quickly.
- **Adaptive Interface:** Consider adding user-level adaptability to display simpler or more complex features based on user proficiency, thus accommodating both novice and expert contributors.

4.5.2 Suggestions to Improve Performance Requirements

- **Cache Mechanisms:** Implement caching strategies for repeated video access and model predictions to reduce redundant computations and accelerate load times.
- **Parallel Data Processing:** Adopt parallel processing or distributed computing for resource-intensive tasks such as large-scale video conversions or batch inference jobs, making full use of modern HPC architectures.

- **Scalability Testing:** Periodically perform stress tests to identify bottlenecks under high user load or large data volumes; these tests help guide hardware upgrades or optimizations in system architecture.

4.5.3 Suggestions to Improve Reliability Requirements

- **Redundant Backups:** Schedule automated backups of both raw video and annotated data to off-site or cloud storage to mitigate data loss risks.
- **Health Monitoring:** Implement monitoring tools (e.g., Prometheus, Grafana) to track system metrics and quickly identify anomalies or potential failures.
- **Failover Strategy:** Define procedures to switch to backup systems or mirror databases in the event of primary component failure, minimizing downtime for users.

4.5.4 Suggestions to Improve Maintainability Requirements

- **Modular Code Reviews:** Implement a peer review process where each module is reviewed by multiple developers, ensuring code quality and knowledge sharing.
- **Design Patterns and Best Practices:** Standardize architectures (e.g., microservices) and coding guidelines to ensure consistent implementations across modules.

4.5.5 Suggestions to Improve Portability Requirements

- **Containerization:** Package core components (database, web front-end, machine learning services) in containers (e.g., Docker) to streamline deployments across multiple environments.
- **Cross-Browser Testing:** Continuously validate that the interface renders correctly on major web browsers (e.g., Chrome, Firefox, Safari, Edge) to avoid unexpected UI/UX issues.

- **Adaptive UI Design:** Ensure the front-end is responsive and supports diverse screen resolutions, enabling efficient use on tablets, smartphones, and various display sizes.

4.6 Design Constraints

1. **Enhanced Standards Compliance and Data Interoperability**

    - **Adopt Additional Metadata Standards:** In addition to Darwin Core (DwC), we can incorporate widely used schemas (e.g., ABCD, MIxS) or extend DwC fields to capture more granular information (e.g., habitat type, light levels, or ROV operational details). This promotes seamless integration with other marine datasets and expands future analytical potential.
    - **Strengthen Versioning Protocols:** Implement version-control systems for both raw and annotated datasets to ensure provenance is fully traceable. Maintaining a structured change log will allow researchers to track and cite specific data revisions confidently.

2. **Open-Source Software and Reproducibility**

    - **Adopt a Community-Driven Development Model:** Encourage external contributions to core modules (e.g., machine learning pipelines, data ingestion scripts) by using a transparent governance approach such as Requests for Comment (RFC) on public code repositories. This fosters a broader support network and quicker integration of cutting-edge methods.

3. **Strengthened Ethical and Regulatory Compliance**

    - **Automate Compliance Checks:** Integrate compliance audits (GDPR, Zooniverse terms) into the development pipeline. For example, script-based checks can verify that no personal data is stored, that consent is properly documented, and that data usage remains within ethical guidelines.

- **Incorporate Accessibility Standards:** Ensure the citizen science interface follows accessibility guidelines (e.g., WCAG 2.1). This broadens volunteer inclusivity and may be required by funders and institutions advocating for accessible digital environments.

4. **Future-Proofing Hardware and Computational Constraints**

   - **Design for Scalable Architecture:** Although current GPU and CPU resources may be sufficient, plan for seamless scaling to larger or distributed clusters if future data volumes or model complexity outgrow on-site capacity. Containerization (e.g., Docker, Kubernetes) can facilitate portability and load balancing across multiple machines.
   - **Implement Resource-Aware Model Training:** Employ model-pruning, quantization, or adaptive inference strategies that reduce memory consumption without substantially impacting accuracy. This ensures the system remains efficient under varying hardware constraints.

5. **Comprehensive Data Sharing and Institutional Coordination**

   - **Harmonize Institutional and National Policies:** We should create a unified framework that addresses both Swedish national regulations and individual institutional policies at the University of Gothenburg and Chalmers University of Technology. Clearly define roles, responsibilities, and escalation procedures for compliance and conflict resolution.
   - **Facilitate Cross-Project Integrations:** Establish formalized Application Programming Interfaces (APIs) or data exchange protocols that allow other marine observatories or interdisciplinary projects to query, visualize, and annotate the Koster Seafloor Observatory's data. This fosters collaboration and ensures the repository's long-term relevance.

4.7 Supporting Information

The recommended improvements ensure that the Koster Seafloor Observatory can seamlessly support a diverse user community, handle increasingly large datasets, and remain adaptable over time. By simplifying the user interface and automating tasks, the platform becomes more accessible to both citizen scientists and researchers. Optimized data transfer, real-time machine learning responses, and failover strategies enhance performance and reliability, while modular architecture and continuous integration practices foster easy maintenance and growth. Finally, standards-based design, containerization, and cross-device compatibility secure the platform's portability across varied computational environments.